

1 Introduction

This report explores the implementation of distributed memory parallelism to enhance a computer program by employing domain decomposition with MPI. The task involves adding the necessary MPI calls to enable domain decomposition parallelism in a program designed to simulate a 2D random field Ising model.

It should be noted that to change the number of processors our code runs with, we take the approach of setting `--ntasks-per-node=P` where P is the number of processors we demand. This is because we are running our code on the PX457 partition which has 2 dedicated nodes at any given time each with at most 48 processors but we will only be using a maximum of 25 processors for any test run of our code.

We begin by compiling the original code using the `Makefile` provided and running the `make` command. Then, in our `avon` home directory, we push a job into the queue using the `rfim.slurm` file. The output is recorded below:

Output
Size of each local processor grid : 480 x 480
Global magnetisation at cycle 0 : 0.001172 Global
magnetisation at cycle 100 : 0.007569 Global
magnetisation at cycle 200 : 0.020547 Global
magnetisation at cycle 300 : 0.023186 Global
magnetisation at cycle 400 : 0.023290 Global
magnetisation at cycle 500 : 0.028247 Global
magnetisation at cycle 600 : 0.033958 Global
magnetisation at cycle 700 : 0.032517 Global
magnetisation at cycle 800 : 0.033932 Global
magnetisation at cycle 900 : 0.031745 Global
magnetisation at cycle 1000 : 0.030156 End of simulation. Rank 0 accepted 3026981 moves.

Table 1: Summary of original code results with no MPI

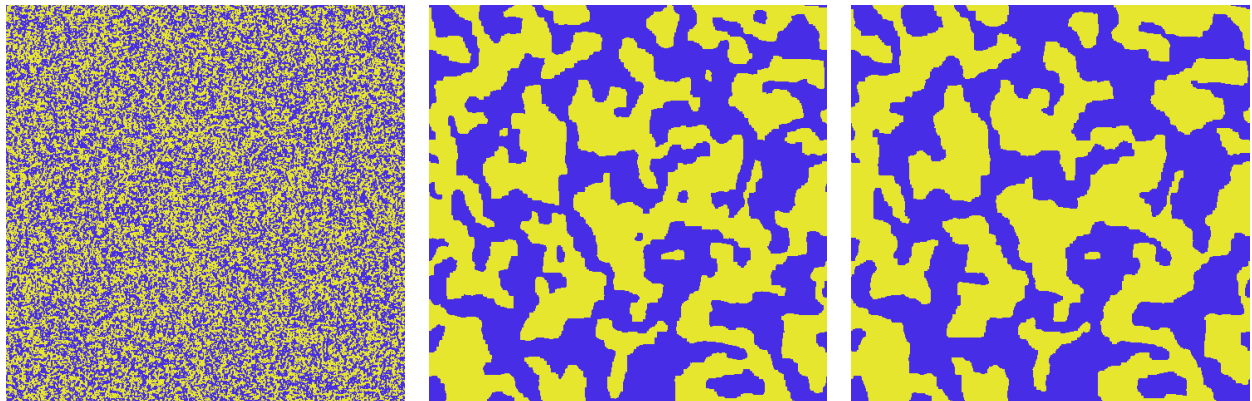


Figure 1: Global magnetisation grid at cycles 0, 600 and 1000 for the original code.

1.1 Setting up MPI

Next, we make the modifications to `comms_initialise` in the `comms.c` file, by adding appropriate calls to initialise MPI, store the rank of the current processor, and the size of the communicator. We also use the `MPI_Wtime()` command across the `comms_initialise` and `comms_finalise` routines to compute the time elapsed. We request 4 processors from `avon`. This makes the program run substantially faster. In our `avon` home directory, we run this modified code by pushing it as a job into the queue and display the corresponding output below:

Output
Size of each local processor grid : 240 x 240
Global magnetisation at cycle 0 : 0.003845
Global magnetisation at cycle 100 : 0.038646
Global magnetisation at cycle 200 : 0.052378
Global magnetisation at cycle 300 : 0.055720
Global magnetisation at cycle 400 : 0.060512
Global magnetisation at cycle 500 : 0.062144
Global magnetisation at cycle 600 : 0.066814
Global magnetisation at cycle 700 : 0.070573
Global magnetisation at cycle 800 : 0.071632
End of simulation. Rank 1 accepted 775406 moves.
End of simulation. Rank 3 accepted 749998 moves.
Global magnetisation at cycle 900 : 0.074913
End of simulation. Rank 2 accepted 728889 moves.
Global magnetisation at cycle 1000 : 0.078012
End of simulation. Rank 0 accepted 753250
moves. Total time elapsed since MPI initialised
: 3.533528 s

Table 2: Summary of original code results with MPI

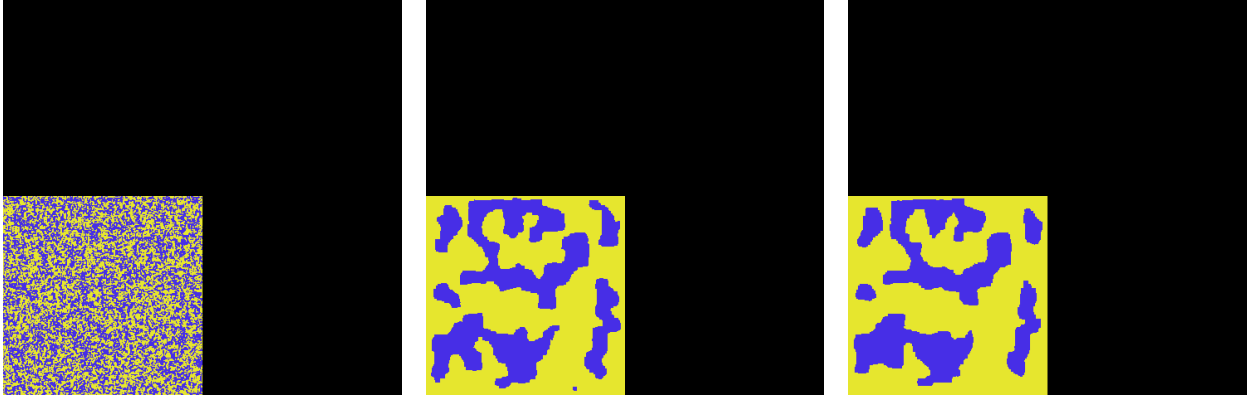


Figure 2: Global magnetisation grid at cycles 0, 600 and 1000 for the original code with MPI initialisation on 4 processors.

Indeed, we observe that each processor has a local size of 240×240 which is one quarter of the original domain size. However, in the final output image, we observe only a quarter of the grid is printed. This is because each of the four processors is simulating the same quarter of the grid independently, without sharing the workload. To address this issue, we need to create communication between the processors so they can divide the work evenly across different regions of the grid and then combine the results into a single unified image.

1.2 Sharing the work - Setting up a Cartesian topology

We modify `comms_processor_map` to set up the `my_rank_coords` array appropriately so that each processor works on a different part of the grid. Currently this is set to $(0,0)$ for all MPI tasks. We use `MPI_Comm_cart_comm` to setup a Cartesian communicator. Using the predefined variables, we call `MPI_Cart_create(MPI_COMM_WORLD, ndims, dims, pbc, reorder, cart_comm)` which creates our Cartesian communicator. Then, we use `MPI_Cart_shift` to store the neighbours of the current MPI task into the array `my_rank_neighbours`. We make use of the predefined constants in `grid.c`, namely `left`, `right`, `up` and `down` to reference the neighbours. The periodic boundary conditions are taken care for by the `pbc` variable passed before into the Cartesian communicator. We add debugging print statements which outputs the contents of the arrays `my_rank_coords` and `my_rank_neighbours` in each MPI task. We execute this on 1, 4 and 9 processors on `avon` and output the debugging print statements for each run on the next 3 pages:

Processors	Output
1	<pre> MPI Rank 0: my_rank_coords = (0, 0), MPI Rank 0: my_rank_neighbours = [left: 0, right: 0, down: 0, up: 0], Size of each local processor grid : 480 x 480, Global magnetisation at cycle 0 : 0.001172, Global magnetisation at cycle 100 : 0.007569, Global magnetisation at cycle 200 : 0.020547, Global magnetisation at cycle 300 : 0.023186, Global magnetisation at cycle 400 : 0.023290, Global magnetisation at cycle 500 : 0.028247, Global magnetisation at cycle 600 : 0.033958, Global magnetisation at cycle 700 : 0.032517, Global magnetisation at cycle 800 : 0.033932, Global magnetisation at cycle 900 : 0.031745, Global magnetisation at cycle 1000 : 0.030156, End of simulation. Rank 0 accepted 3026981 moves.</pre>

Table 3: Output of modified code with a Cartesian topology running on 1 processor.

Processors	Output
4	<pre> MPI Rank 0: my_rank_coords = (0, 0), MPI Rank 0: my_rank_neighbours = [left: 2, right: 2, down: 1, up: 1], Size of each local processor grid : 240 x 240, Global magnetisation at cycle 0 : 0.003845, MPI Rank 1: my_rank_coords = (0, 1), MPI Rank 1: my_rank_neighbours = [left: 3, right: 3, down: 0, up: 0], MPI Rank 2: my_rank_coords = (1, 0), MPI Rank 2: my_rank_neighbours = [left: 0, right: 0, down: 3, up: 3], MPI Rank 3: my_rank_coords = (1, 1), MPI Rank 3: my_rank_neighbours = [left: 1, right: 1, down: 2, up: 2], Global magnetisation at cycle 100 : 0.038646, Global magnetisation at cycle 200 : 0.052378, Global magnetisation at cycle 300 : 0.055720, Global magnetisation at cycle 400 : 0.060512, Global magnetisation at cycle 500 : 0.062144, Global magnetisation at cycle 600 : 0.066814, Global magnetisation at cycle 700 : 0.070573, Global magnetisation at cycle 800 : 0.071632, Global magnetisation at cycle 900 : 0.074913, Global magnetisation at cycle 1000 : 0.078012, End of simulation. Rank 2 accepted 728889 moves. End of simulation. Rank 3 accepted 749998 moves. End of simulation. Rank 1 accepted 775406 moves. End of simulation. Rank 0 accepted 753250 moves. Total time elapsed since MPI initialised : 3.321770 s. </pre>

Table 4: Output of modified code with a Cartesian topology running on 4 processors.

Processors	Output
9	<pre> MPI Rank 0: my_rank.coords = (0, 0), MPI Rank 0: my_rank.neighbours = [left: 6, right: 3, down: 2, up: 1], Size of each local processor grid : 160 x 160, Global magnetisation at cycle 0 : 0.001450, MPI Rank 1: my_rank.coords = (0, 1), MPI Rank 1: my_rank.neighbours = [left: 7, right: 4, down: 0, up: 2], MPI Rank 2: my_rank.coords = (0, 2), MPI Rank 2: my_rank.neighbours = [left: 8, right: 5, down: 1, up: 0], MPI Rank 3: my_rank.coords = (1, 0), MPI Rank 3: my_rank.neighbours = [left: 0, right: 6, down: 5, up: 4], MPI Rank 4: my_rank.coords = (1, 1), MPI Rank 4: my_rank.neighbours = [left: 1, right: 7, down: 3, up: 5], MPI Rank 5: my_rank.coords = (1, 2), MPI Rank 5: my_rank.neighbours = [left: 2, right: 8, down: 4, up: 3], MPI Rank 6: my_rank.coords = (2, 0), MPI Rank 6: my_rank.neighbours = [left: 3, right: 0, down: 8, up: 7], MPI Rank 7: my_rank.coords = (2, 1), MPI Rank 7: my_rank.neighbours = [left: 4, right: 1, down: 6, up: 8], MPI Rank 8: my_rank.coords = (2, 2), MPI Rank 8: my_rank.neighbours = [left: 5, right: 2, down: 7, up: 6], Global magnetisation at cycle 100 : 0.015816, Global magnetisation at cycle 200 : 0.021458, Global magnetisation at cycle 300 : 0.023194, Global magnetisation at cycle 400 : 0.024019, Global magnetisation at cycle 500 : 0.024297, Global magnetisation at cycle 600 : 0.023819, Global magnetisation at cycle 700 : 0.023993, End of simulation. Rank 7 accepted 303911 moves., End of simulation. Rank 6 accepted 319311 moves., End of simulation. Rank 1 accepted 304328 moves., End of simulation. Rank 2 accepted 278712 moves., End of simulation. Rank 3 accepted 322690 moves., End of simulation. Rank 4 accepted 302669 moves., End of simulation. Rank 5 accepted 289767 moves., End of simulation. Rank 8 accepted 319958 moves., Global magnetisation at cycle 800 : 0.024375, Global magnetisation at cycle 900 : 0.025035, Global magnetisation at cycle 1000 : 0.024748, End of simulation. Rank 0 accepted 310745 moves., Total time elapsed since MPI initialised : 1.697735 s </pre>

Table 5: Output of modified code with a Cartesian topology running on 9 processors.

From the debugging print statements, we observe that each processor is working on a distinct part of the grid, and the neighbors are set up correctly to ensure that the periodic boundary conditions are implemented as intended. The MPI calls used to achieve this were: `MPI_Cart_create`, `MPI_Cart_coords`, and `MPI_Cart_shift`.

`MPI_Cart_create` - This function creates a Cartesian topology for a communicator. It is useful for us to communicate with neighbours much more easily and we can make use of its neighbouring functions rather than resorting to modular arithmetic to setup periodic boundary conditions by hands.

`MPI_Cart_coords` - This function determines the Cartesian coordinates of a specific processor in the grid. It is useful for knowing which subdomain a specific processor is working on.

`MPI_Cart_shift` - This function determines the neighbouring processors in the topology without us having to do any explicit calculations. This is especially useful in higher dimensions when such calculations can become tedious.

1.3 Collecting data from all processors

Next, we modify `comms_get_global_grid` to collect the array `grid_spin` which contains the subdomain information for each processor and assemble them into one array called `global_grid_spin` on the rank 0 processor. This will allow us to collect the work from each processor and combine it onto a single processor. We make use of `MPI_Send` and `MPI_Recv` commands to achieve the communication of data from other processors to the rank 0 processor. We make use of `tags` to avoid any confusion of messages from different ranks. Furthermore, we modify the `comms_get_global_mag` routine by adding a reduction MPI collective routine which allows us to average each value of `local_mag` into `global_mag` on the rank 0 MPI process, which then outputs this already in the main program. The following table displays the results after implementing the aforementioned changes to the code. It is ran on 4 processors.

Output
<pre> MPI Rank 0: my_rank_coords = (0, 0) MPI Rank 0: my_rank_neighbours = [left: 2, right: 2, down: 1, up: 1] Size of each local processor grid : 240 x 240 MPI Rank 1: my_rank_coords = (0, 1) MPI Rank 1: my_rank_neighbours = [left: 3, right: 3, down: 0, up: 0] MPI Rank 2: my_rank_coords = (1, 0) MPI Rank 2: my_rank_neighbours = [left: 0, right: 0, down: 3, up: 3] MPI Rank 3: my_rank_coords = (1, 1) MPI Rank 3: my_rank_neighbours = [left: 1, right: 1, down: 2, up: 2] Global magnetisation at cycle 0 : 0.010668 Global magnetisation at cycle 100 : 0.115339 Global magnetisation at cycle 200 : 0.139106 Global magnetisation at cycle 300 : 0.150859 Global magnetisation at cycle 400 : 0.163142 Global magnetisation at cycle 500 : 0.171953 Global magnetisation at cycle 600 : 0.183438 Global magnetisation at cycle 700 : 0.194418 Global magnetisation at cycle 800 : 0.201849 Global magnetisation at cycle 900 : 0.208681 Global magnetisation at cycle 1000 : 0.216979 End of simulation. Rank 1 accepted 775406 moves. End of simulation. Rank 2 accepted 728889 moves. End of simulation. Rank 3 accepted 749998 moves. End of simulation. Rank 0 accepted 753250 moves. Total time elapsed since MPI initialised : 3.922214 s </pre>

Table 6: Output of modified code with data collection from all MPI processes and magnetisation averaging.

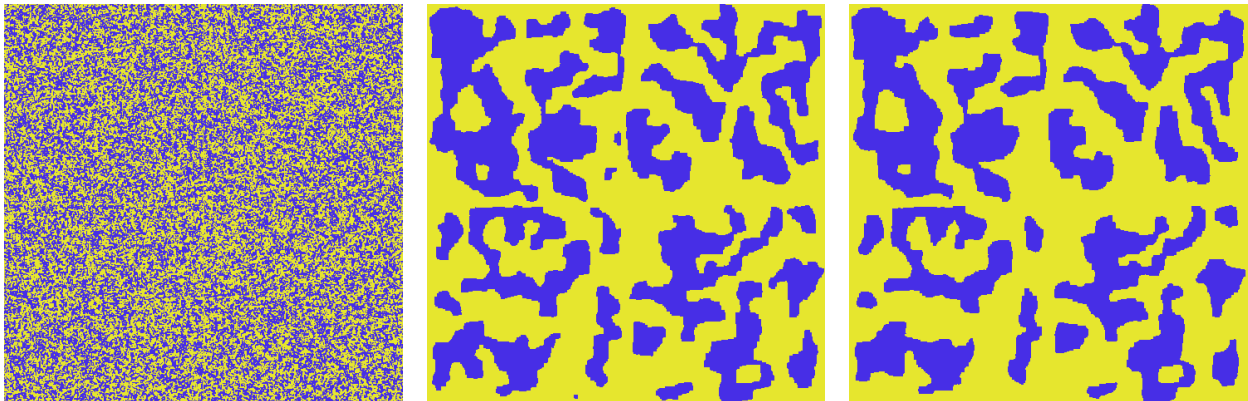


Figure 3: Global magnetisation grid at cycles 0, 600 and 1000 for the newly modified code with data collection from each processor.

The image above demonstrates that we have successfully gathered the results from each processor's work on its respective subdomain and combined them to form the complete output. However, a distinct yellow border is visible around each subdomain. This border results from the lack of communication between processors. To eliminate this issue, we need communication between neighboring processors so that boundary values can be correctly exchanged. This ensures that the subdomains are properly connected, resulting in a continuous and accurate final output.

1.4 Halo Swapping

When we use multiple processor's to handle different areas of the grid, the boundaries of these subdomains will appear incorrectly since they require information from the boundaries of neighbouring subdomains. Therefore, we need to send and receive boundary information between different processors. To fix this, we go to the `comms.halo_swap` function and for each boundary (left, right, down, up) we append to `sendbuf` the appropriate boundary elements from `grid_spin` (the local subdomain to each processor).

```
for (iy = 0; iy < grid_domain_size; iy++) {
    sendbuf[iy] = grid_spin[iy][0];
}
```

Then, we use `MPI_Sendrecv` to send one boundary and receive another at the same time.

```
MPI_Sendrecv(sendbuf, grid_domain_size, MPI_INT, my_rank_neighbours[left], 0,
             recvbuf, grid_domain_size, MPI_INT, my_rank_neighbours[right], 0,
             MPI_COMM_WORLD, &status);
```

The boundary that is received is stored in `grid_halo` as follows:

```
for (iy = 0; iy < grid_domain_size; iy++) {
    grid_halo[right][iy] = recvbuf[iy];
}
```

which is used to correctly compute the boundary on the subdomain associated with the receiving processor since it now has access to the neighbouring processors boundary data.

We then run the resulting code on 4 processors on `avon` and the results are shown below:

Output
<pre> MPI Rank 0: my_rank_coords = (0, 0) MPI Rank 0: my_rank_neighbours = [left: 2, right: 2, down: 1, up: 1] Size of each local processor grid : 240 x 240 MPI Rank 1: my_rank_coords = (0, 1) MPI Rank 1: my_rank_neighbours = [left: 3, right: 3, down: 0, up: 0] MPI Rank 2: my_rank_coords = (1, 0) MPI Rank 2: my_rank_neighbours = [left: 0, right: 0, down: 3, up: 3] MPI Rank 3: my_rank_coords = (1, 1) MPI Rank 3: my_rank_neighbours = [left: 1, right: 1, down: 2, up: 2] Global magnetisation at cycle 0 : 0.004375 Global magnetisation at cycle 100 : 0.032161 Global magnetisation at cycle 200 : 0.037552 Global magnetisation at cycle 300 : 0.044262 Global magnetisation at cycle 400 : 0.042778 Global magnetisation at cycle 500 : 0.041953 Global magnetisation at cycle 600 : 0.046311 Global magnetisation at cycle 700 : 0.053759 Global magnetisation at cycle 800 : 0.057431 Global magnetisation at cycle 900 : 0.059236 Global magnetisation at cycle 1000 : 0.066493 End of simulation. Rank 1 accepted 794655 moves. End of simulation. Rank 2 accepted 747481 moves. End of simulation. Rank 3 accepted 806864 moves. End of simulation. Rank 0 accepted 783269 moves. Total time elapsed since MPI initialised : 4.155231 s </pre>

Table 7: Output of modified code with halo swapping implemented

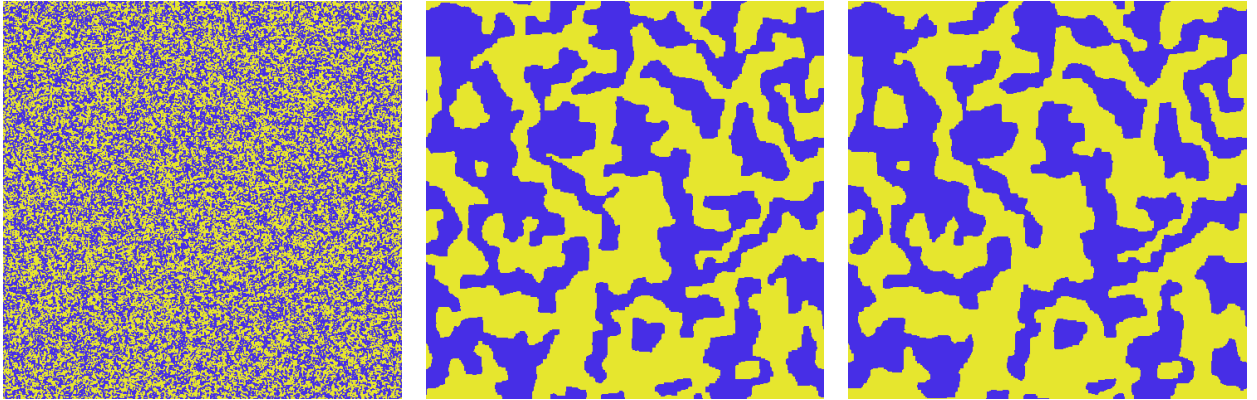


Figure 4: Global magnetisation grid at cycles 0, 600 and 1000 for the modified code.

In conclusion, to achieve a fully functional domain decomposed parallel simulation, we firstly initialised MPI then setup a Cartesian topology to decompose our domain into multiple regions, each dedicated to one processor. We made sure this was correctly setup with some debugging statements. Then, we collected the computations done on each processor into the rank 0 processor and correctly allocated each `grid_spin` (local domain) into the overall domain, `global_grid_spin`. Then we set up a halo swapping function which allowed us to send and receive boundary information across processors to correctly compute boundaries on each processors subdomain. This achieved our domain decomposition using MPI.

1.5 Timings and report

We now experiment with timings of our program. For these benchmark tests, we comment lines 217 – 223 in `rfim.c` to avoid printing images so we can compare the true computational timings of our parallelised code on 1, 4, 9, 16, 25 processors on `avon` using grid sizes 480, 600, 720, 840.

(Note: the `comms_finalise` function of the `comms.c` file had an if statement with a clause that if `p > 1` then it outputs the execution time. We have removed this so we can time the program with exactly 1 processor).

Grid Size	1 Processor	4 Processors	9 Processors	16 Processors	25 Processors
480	24.329109	5.525788	3.365646	1.209924	4.781192
	22.282765	3.186897	3.070991	0.704989	0.465892
	23.796125	3.145397	2.824815	3.790893	1.375653
600	40.322465	7.147961	4.107423	3.897143	5.309618
	41.094191	7.328104	2.184956	4.445129	0.742719
	45.729198	6.097492	3.914220	4.281198	2.426909
720	66.012584	9.079527	5.064220	1.777059	1.082434
	56.226336	11.609030	5.003574	2.196158	5.811449
	61.435313	11.624713	5.091090	4.711243	1.088911
840	95.552807	15.721693	7.383254	5.338715	2.673775
	79.632213	14.972235	6.983544	5.636353	6.324616
	88.101485	15.411972	7.183578	5.387478	2.934973

Table 8: Program timings with variable processors and grid sizes

We observe there is some variable timings on repeats for some of these tests, this could be due to high demands for resources on `avon` so we average the results to product the following table:

Grid Size	1 Processor	4 Processors	9 Processors	16 Processors	25 Processors
480	23.13633	3.952361	3.087817	1.901602	2.207579
600	42.048618	6.524853	3.402866	4.207823	2.493082
720	61.224411	10.437423	5.053294	2.561820	2.327265
840	87.095835	15.368300	7.183459	5.454182	3.644788

Table 9: Averaged program timings with variable processors and grid sizes

We also wish to compute a speed up function ψ as a function of P , the number of processors, for each grid size. The formula for ψ is given as

$$\psi(P) = \frac{T(1)}{T(P)}$$

where $T(P)$ is the time taken to run with P MPI tasks, and $T(1)$ is the time taken with 1 MPI task, so that $\psi(1) = 1.0$ exactly.

Grid Size	$\psi(1)$	$\psi(4)$	$\psi(9)$	$\psi(16)$	$\psi(25)$
480	1.0	5.853799	7.492779	12.166757	10.480408
600	1.0	6.444377	12.356825	9.992963	16.866119
720	1.0	5.865855	12.115742	23.898795	26.307451
840	1.0	5.667239	12.124498	15.968633	23.895995

Table 10: Speed up function, ψ , for each grid size

The most clear observation is that for any given grid size, the speed up function is clearly increasing as the number of processors increases as expected by the parallelism. Also, for grid size 480 we observe that performance actually decreases when increasing the processors from 16 to 25 which could be perhaps because the overhead from managing a large number of processors outweighs the benefits of decomposing the domain into smaller subdomains. This idea is observed best when we compare the speed up from 1 to 4 processors against the speed up from 16 to 25 processors because the increase in the former is roughly a multiplier of 6 where as in the latter the increase is roughly a multiplier of 1.5.

Processors	Global Grid Size	Local Grid Size	Rank coordinates and neighbours	
1	1.0	5.853799	7.492779	12.166757
4	1.0	6.444377	12.356825	9.992963
9	1.0	5.865855	12.115742	23.898795
16	1.0	5.667239	12.124498	15.968633
25	height			

Table 11: Speed up function, ψ , for each grid size